Task 3: College Club Membership Management

Task1 : Identify Data Problems

3.1: Identify Data Problems

Here the given data is not in systematic form and there are some problems. Some of the problems in the data are discussed below:

a. Data Redundancy

The same students data repeated multiple times. For example, Asha (StudentID 1) is shown twice since she joined two clubs. Her email address and name are repeated. Bikash and Nisha experience the same thing. As a result, the table has needless repetition. It increases storage use and also makes the table messy.

b. Update Anomaly

Details of the club, including ClubRoom and ClubMentor, are frequently repeated. For instance, "Music Club – R101 – Mr. Raman" appears more than once for several students. If a club mentor changes, the update must be done in many rows, which easily cause the error.

c. Insertion Anomaly

An insertion issue is brought about by the table structure. A minimum of one student must join a club before it may be added. For instance, without adding a student record, we are unable to add the room number and mentor of a newly formed "Art Club" to this database. This suggests that we need to enter unnecessary or missing student data in order to store club information.

Let's explain why this table is not normalized.

Various kinds of information are combined in one location in this table. Although they rely on various entities, student, club, and membership information are all kept in one table. Although they are not correctly separated, club information depends on the ClubName and student information depends on the StudentID. Because of this poor structure, the table contains problems like data redundancy and other data anomalies. As a result, the table does not follow to the proper normalization criteria.

3. Key Definitions:

- ➢ Redundant data : This refers to storing same information repeatedly when it is not necessary.
- ➢ Duplicate data : This means the exact same record is stored more than once.
- ➢ Insert anomaly : This mainy happens when it is difficult to add new data due to some other data is missing.
- ➢ Update anomaly : It occurs when the same information has to be changed in many rows, and missing one update can cause incorrect data.
- ➢ Deletion anomaly : It happens when deleting one data also removes import information that should not be lost.

2. Normalize the Data

Steps for complete normalization of the data.
First open the docker and cmd and start to run the following commands.

1. docker

```
C:\Users\Lenovo>docker
Usage:  docker [OPTIONS] COMMAND

A self-sufficient runtime for containers

Common Commands:
  run         Create and run a new container from an image
  exec        Execute a command in a running container
  ps          List containers
  build       Build an image from a Dockerfile
  bake        Build from a file
  pull        Download an image from a registry
  push        Upload an image to a registry
  images      List images
```

2. docker images (to check weather mysql is there or not)

```
C:\Users\Lenovo>docker images
REPOSITORY                 TAG       IMAGE ID       CREATED        SIZE
network_reconnaissance     latest    8493677163ae   3 weeks ago    935MB
<none>                     <none>    76b8afe53737   3 weeks ago    847MB
mysql                      8.0       0546f878adee   4 weeks ago    1.07GB
my-custom-ubuntu           v1        ac2849f9d2be   2 months ago   1.11GB
kalilinux/kali-rolling     latest    c8a4573bf14f   3 months ago   192MB
ubuntu                     latest    c35e29c94501   4 months ago   117MB
docker/welcome-to-docker   latest    c4d56c24da4f   6 months ago   22.2MB
```

3. docker pull mysql

```
C:\Users\Lenovo>docker pull mysql:8.0
8.0: Pulling from library/mysql
bde62e757594: Pull complete
ce98f3559366: Pull complete
f508d7fab5b3: Pull complete
e7a04c019bde: Pull complete
bae900376130: Pull complete
4f37333d1be6: Pull complete
a9a9deeee02a: Pull complete
e05db5310ebc: Pull complete
2e2c1f6f8d57: Pull complete
d442b2c1726e: Pull complete
23fbf4028535: Pull complete
Digest: sha256:99d774bf02a48a1bb1c599920d2571946d31e5940b854b02737d5e95c184358f
Status: Downloaded newer image for mysql:8.0
docker.io/library/mysql:8.0
```

4. docker run --name college-mysql -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=collegeDB -p 3307:3306 -d mysql:8.0

```
C:\Users\Lenovo>docker run --name college-mysql -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=collegeDB -p 3307:3306 -d mysql:8.0
ced45c8253489833cd9adb7713155e4a8aef6593c61fd1dec044ab6fe5153c05
```

5. docker exec -it college-mysql mysql -u root -p (we have to enter password here and the password is root)

```
C:\Users\Lenovo>docker exec -it college-mysql mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.45 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.
```

After this the process for creating the database and normalizing it starts:

Creating 1NF Table

Every database record must be unique, and each column must only contain atomic (single) values, according to First Normal Form (1NF). Every data is inserted and 1NF table is created and it looks like the below.

```
mysql> USE collegeDB;
Database changed
mysql> CREATE TABLE StudentClub_1NF (
    ->     StudentID INT,
    ->     StudentName VARCHAR(50),
    ->     Email VARCHAR(100),
    ->     ClubName VARCHAR(50),
    ->     ClubRoom VARCHAR(10),
    ->     ClubMentor VARCHAR(50),
    ->     JoinDate DATE,
    ->     PRIMARY KEY (StudentID, ClubName)
    -> );
Query OK, 0 rows affected (0.10 sec)
```

```
mysql> INSERT INTO StudentClub_1NF VALUES
    -> (1,'Asha','asha@email.com','Music Club','R101','Mr. Raman','2024-01-10'),
    -> (2,'Bikash','bikash@email.com','Sports Club','R202','Ms. Sita','2024-01-12'),
    -> (1,'Asha','asha@email.com','Sports Club','R202','Ms. Sita','2024-01-15'),
    -> (3,'Nisha','nisha@email.com','Music Club','R101','Mr. Raman','2024-01-20'),
    -> (4,'Rohan','rohan@email.com','Drama Club','R303','Mr. Kiran','2024-01-18'),
    -> (5,'Suman','suman@email.com','Music Club','R101','Mr. Raman','2024-01-22'),
    -> (2,'Bikash','bikash@email.com','Drama Club','R303','Mr. Kiran','2024-01-25'),
    -> (6,'Pooja','pooja@email.com','Sports Club','R202','Ms. Sita','2024-01-27'),
    -> (3,'Nisha','nisha@email.com','Coding Club','Lab1','Mr. Anil','2024-01-28'),
    -> (7,'Aman','aman@email.com','Coding Club','Lab1','Mr. Anil','2024-01-30');
Query OK, 10 rows affected (0.09 sec)
Records: 10  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM StudentClub_1NF;
+-----------+-------------+------------------+-------------+----------+------------+------------+
| StudentID | StudentName | Email            | ClubName    | ClubRoom | ClubMentor | JoinDate   |
+-----------+-------------+------------------+-------------+----------+------------+------------+
|         1 | Asha        | asha@email.com   | Music Club  | R101     | Mr. Raman  | 2024-01-10 |
|         1 | Asha        | asha@email.com   | Sports Club | R202     | Ms. Sita   | 2024-01-15 |
|         2 | Bikash      | bikash@email.com | Drama Club  | R303     | Mr. Kiran  | 2024-01-25 |
|         2 | Bikash      | bikash@email.com | Sports Club | R202     | Ms. Sita   | 2024-01-12 |
|         3 | Nisha       | nisha@email.com  | Coding Club | Lab1     | Mr. Anil   | 2024-01-28 |
|         3 | Nisha       | nisha@email.com  | Music Club  | R101     | Mr. Raman  | 2024-01-20 |
|         4 | Rohan       | rohan@email.com  | Drama Club  | R303     | Mr. Kiran  | 2024-01-18 |
|         5 | Suman       | suman@email.com  | Music Club  | R101     | Mr. Raman  | 2024-01-22 |
|         6 | Pooja       | pooja@email.com  | Sports Club | R202     | Ms. Sita   | 2024-01-27 |
|         7 | Aman        | aman@email.com   | Coding Club | Lab1     | Mr. Anil   | 2024-01-30 |
+-----------+-------------+------------------+-------------+----------+------------+------------+
10 rows in set (0.01 sec)
```

A single column does not have more than one value (for instance, no column has two club names in a single cell). As a result, the table meets the 1NF atomic value requirement.

Identifying the primary key.

A column (or set of columns) that uniquely identifies every entry in a table is called a primary key. StudentID cannot be the primary key in this table since a student may appear more than once when they join other clubs. We require a composite primary key in order to uniquely identify every record.

The following would be a suitable primary key: ClubName + StudentID

Creating 2NF Table

For more effective and rich database normalization, the Second Normal Form (2NF) states that if a table's all columns are at least dependent on a primary key then it can represent 2D data. This means we should remove partial dependency. This implies that we must eliminate any partial dependency. In your given table we used composite primary key (StudentID + ClubName). However, because some attributes depend on ClubName alone (examples: ClubRoom, ClubMentor) while others depend only on StudentID (examples:Student Name and Email), FDs are partial functional dependencies. The table does not meet 2NF because of this partial reliance.

```
mysql> CREATE TABLE Student (
    ->     StudentID INT PRIMARY KEY,
    ->     StudentName VARCHAR(50),
    ->     Email VARCHAR(100)
    -> );
Query OK, 0 rows affected (0.13 sec)
```

```
mysql> INSERT INTO Student
    -> SELECT DISTINCT StudentID, StudentName, Email
    -> FROM StudentClub_1NF;
Query OK, 7 rows affected (0.08 sec)
Records: 7  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM Student;
+-----------+-------------+------------------+
| StudentID | StudentName | Email            |
+-----------+-------------+------------------+
|         1 | Asha        | asha@email.com   |
|         2 | Bikash      | bikash@email.com |
|         3 | Nisha       | nisha@email.com  |
|         4 | Rohan       | rohan@email.com  |
|         5 | Suman       | suman@email.com  |
|         6 | Pooja       | pooja@email.com  |
|         7 | Aman        | aman@email.com   |
+-----------+-------------+------------------+
7 rows in set (0.00 sec)
```

Club

```
mysql> CREATE TABLE Club (
    ->     ClubName VARCHAR(50) PRIMARY KEY,
    ->     ClubRoom VARCHAR(10),
    ->     ClubMentor VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.06 sec)
```

```
mysql> INSERT INTO Club
    -> SELECT DISTINCT ClubName, ClubRoom, ClubMentor
    -> FROM StudentClub_1NF;
Query OK, 4 rows affected (0.08 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

Membership

```
mysql> CREATE TABLE Membership (
    ->     StudentID INT,
    ->     ClubName VARCHAR(50),
    ->     JoinDate DATE,
    ->     PRIMARY KEY (StudentID, ClubName),
    ->     FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
    ->     FOREIGN KEY (ClubName) REFERENCES Club(ClubName)
    -> );
Query OK, 0 rows affected (0.07 sec)
```

```
mysql> INSERT INTO Membership
    -> SELECT StudentID, ClubName, JoinDate
    -> FROM StudentClub_1NF;
Query OK, 10 rows affected (0.08 sec)
Records: 10  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM Membership;
+-----------+-------------+------------+
| StudentID | ClubName    | JoinDate   |
+-----------+-------------+------------+
|         1 | Music Club  | 2024-01-10 |
|         1 | Sports Club | 2024-01-15 |
|         2 | Drama Club  | 2024-01-25 |
|         2 | Sports Club | 2024-01-12 |
|         3 | Coding Club | 2024-01-28 |
|         3 | Music Club  | 2024-01-20 |
|         4 | Drama Club  | 2024-01-18 |
|         5 | Music Club  | 2024-01-22 |
|         6 | Sports Club | 2024-01-27 |
|         7 | Coding Club | 2024-01-30 |
+-----------+-------------+------------+
10 rows in set (0.00 sec)
```

Creating 3NF Table

Third Normal Form (3NF) removes transitive dependencies, enabling each piece of data to depend only on its primary key. Student tables contain information on students, Club tables contain information about clubs, and Membership tables contain information about their relationships. In this manner, the link is managed directly, and student information depends solely on StudentID and club information only on ClubID. It prevents duplication, maintains database organization, and facilitates updates.

```
mysql> CREATE TABLE Student_3NF (
    ->     StudentID INT PRIMARY KEY,
    ->     StudentName VARCHAR(50),
    ->     Email VARCHAR(100) UNIQUE
    -> );
Query OK, 0 rows affected (0.16 sec)
```

```
mysql> INSERT INTO Student_3NF (StudentID, StudentName, Email) VALUES
    -> (1, 'Asha', 'asha@email.com'),
    -> (2, 'Bikash', 'bikash@email.com'),
    -> (3, 'Nisha', 'nisha@email.com'),
    -> (4, 'Rohan', 'rohan@email.com'),
    -> (5, 'Suman', 'suman@email.com'),
    -> (6, 'Pooja', 'pooja@email.com'),
    -> (7, 'Aman', 'aman@email.com');
Query OK, 7 rows affected (0.08 sec)
Records: 7  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM Student_3NF;
+-----------+-------------+------------------+
| StudentID | StudentName | Email            |
+-----------+-------------+------------------+
|         1 | Asha        | asha@email.com   |
|         2 | Bikash      | bikash@email.com |
|         3 | Nisha       | nisha@email.com  |
|         4 | Rohan       | rohan@email.com  |
|         5 | Suman       | suman@email.com  |
|         6 | Pooja       | pooja@email.com  |
|         7 | Aman        | aman@email.com   |
+-----------+-------------+------------------+
7 rows in set (0.00 sec)
```

Club details depend only on ClubID

```
mysql> CREATE TABLE Club_3NF (
    ->     ClubID INT PRIMARY KEY,
    ->     ClubName VARCHAR(50) UNIQUE,
    ->     ClubRoom VARCHAR(20),
    ->     ClubMentor VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.16 sec)
```

```
mysql> INSERT INTO Club_3NF (ClubID, ClubName, ClubRoom, ClubMentor) VALUES
    -> (1, 'Music Club', 'R101', 'Mr. Raman'),
    -> (2, 'Sports Club', 'R202', 'Ms. Sita'),
    -> (3, 'Drama Club', 'R303', 'Mr. Kiran'),
    -> (4, 'Coding Club', 'Lab1', 'Mr. Anil');
Query OK, 4 rows affected (0.08 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM Club_3NF;
+--------+-------------+----------+------------+
| ClubID | ClubName    | ClubRoom | ClubMentor |
+--------+-------------+----------+------------+
|      1 | Music Club  | R101     | Mr. Raman  |
|      2 | Sports Club | R202     | Ms. Sita   |
|      3 | Drama Club  | R303     | Mr. Kiran  |
|      4 | Coding Club | Lab1     | Mr. Anil   |
+--------+-------------+----------+------------+
4 rows in set (0.00 sec)
```

Membership Table 3NF

```
mysql> CREATE TABLE Membership_3NF (
    ->     StudentID INT,
    ->     ClubID INT,
    ->     JoinDate DATE,
    ->     PRIMARY KEY (StudentID, ClubID),
    ->     FOREIGN KEY (StudentID) REFERENCES Student_3NF(StudentID),
    ->     FOREIGN KEY (ClubID) REFERENCES Club_3NF(ClubID)
    -> );
Query OK, 0 rows affected (0.13 sec)
```

```
mysql> INSERT INTO Membership_3NF (StudentID, ClubID, JoinDate) VALUES
    -> (1, 1, '2024-01-10'),
    -> (2, 2, '2024-01-12'),
    -> (1, 2, '2024-01-15'),
    -> (3, 1, '2024-01-20'),
    -> (4, 3, '2024-01-18'),
    -> (5, 1, '2024-01-22'),
    -> (2, 3, '2024-01-25'),
    -> (6, 2, '2024-01-27'),
    -> (3, 4, '2024-01-28'),
    -> (7, 4, '2024-01-30');
Query OK, 10 rows affected (0.07 sec)
Records: 10  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM Membership_3NF;
+-----------+--------+------------+
| StudentID | ClubID | JoinDate   |
+-----------+--------+------------+
|         1 |      1 | 2024-01-10 |
|         1 |      2 | 2024-01-15 |
|         2 |      2 | 2024-01-12 |
|         2 |      3 | 2024-01-25 |
|         3 |      1 | 2024-01-20 |
|         3 |      4 | 2024-01-28 |
|         4 |      3 | 2024-01-18 |
|         5 |      1 | 2024-01-22 |
|         6 |      2 | 2024-01-27 |
|         7 |      4 | 2024-01-30 |
+-----------+--------+------------+
10 rows in set (0.00 sec)
```

ER Diagram

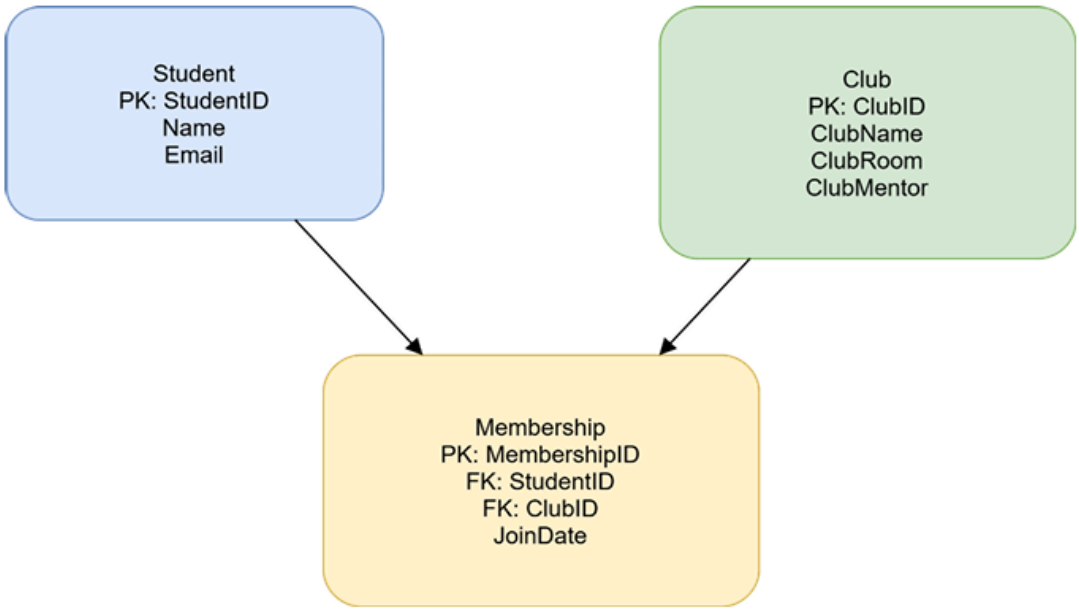

Figure 13: ER Diagram

The student club membership system's entity-relationship diagram shows how data is arranged to remove duplication and facilitate effective administration. Student, Club, and Membership are the three defined entities. The characteristics of the student entity, such as StudentID (primary key), StudentName, and Email, allow each student record to be uniquely recognized. The Club entity's ClubID (primary key), ClubName, ClubRoom, and ClubMentor describe each club independently of its members. With MembershipID as its main key and foreign keys pointing to StudentID and ClubID, the Membership entity serves as a junction table. Additionally, it logs the JoinDate, which documents the moment a student joined a club.

The connections demonstrate that a student may participate in more than one club, and a club may have more than one student. The Membership entity, which connects students and organizations without duplicating information, resolves this many-to-many association. In addition to lowering anomalies and reflecting third normal form (3NF), the model's architecture ensures that student details depend only on StudentID, club information depends only on ClubID, and membership details depend only on MembershipID.